life.augmented

STM32G4内核性能篇

意法半导体微控制器部门

Agenda

1 ART加速

2 CCM SRAM加速

3 CoreMark测试 Vs 主频

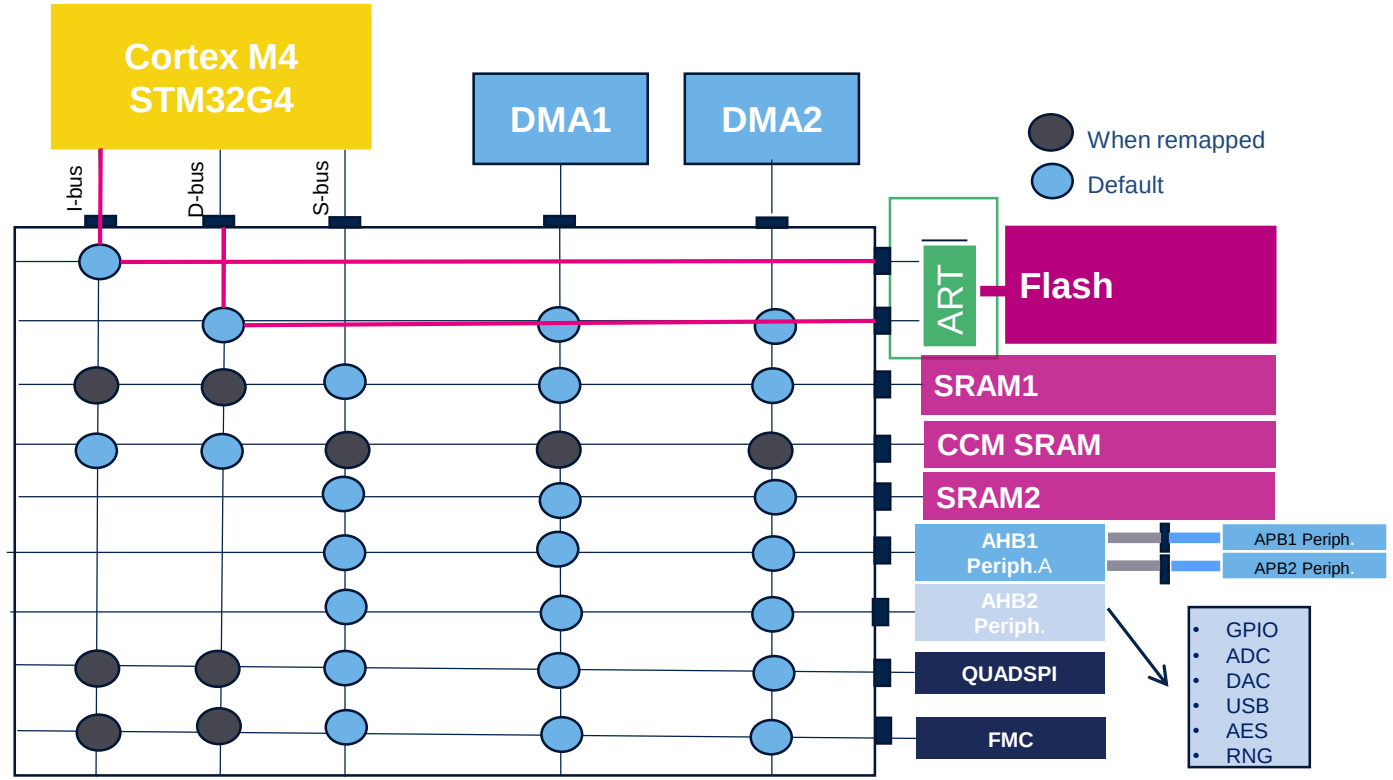
4 乘加指令 & 浮点运算

life.augmented

ART加速

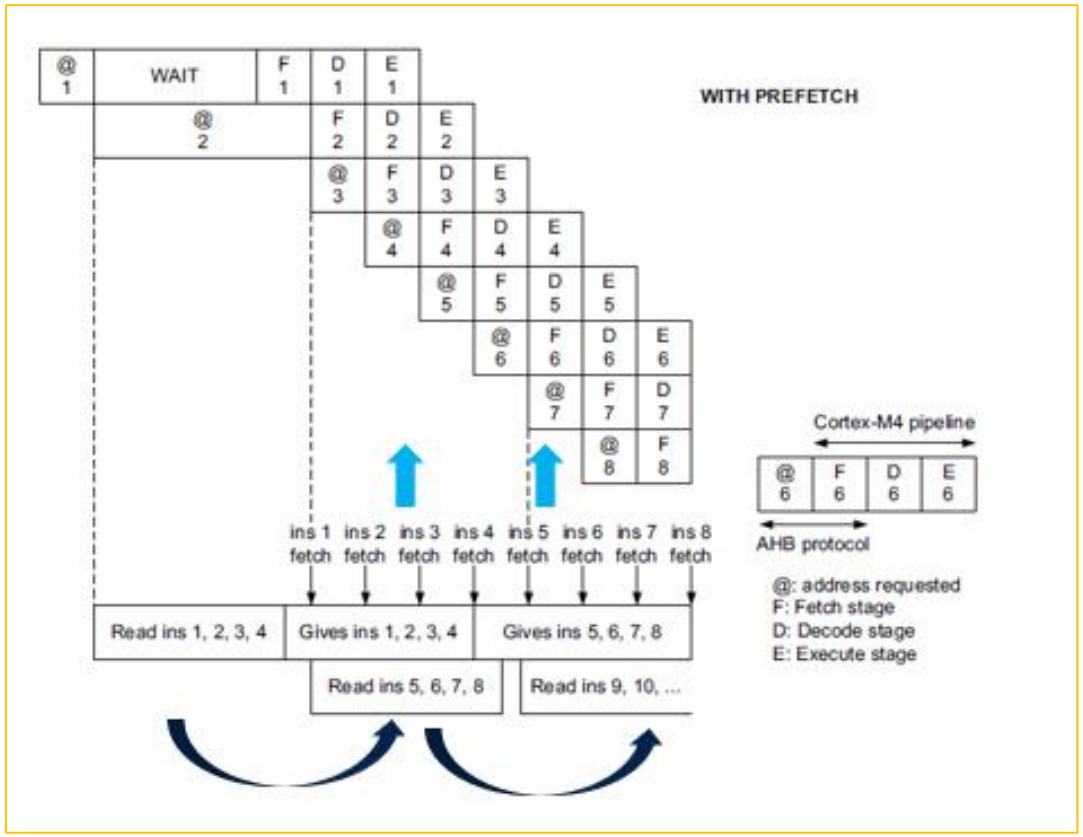
实现运行Flash零等待周期

- 正常运行过程中程序在Flash速度小于主频速度
- 需要特别功能加速Flash数据指令执行
- ART Accelerator™助力运行速度



life.augmented

指令预取机制



- 使能开关：Flash ACR寄存器的PRFTEN位

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBG_SWEN	Res.	Res.
													rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SLEEP_PD	RUN_PD	DCRST	ICRST	DCEN	ICEN	PRFTEN	Res.	Res.	Res.	Res.	LATENCY[3:0]			
	rw	rw	rw	rw	rw	rw	rw					rw	rw	rw	rw

- 程序实现（任选一种）：
  1，使用库函数：__HAL_FLASH_PREFETCH_BUFFER_ENABLE();
  2，寄存器操作： FLASH->ACR |= (1<<8);

Cache

- 指令Cache（I-Cache）
  - 32 lines 2 x 128 bits single bank mode
  - 32 lines of 4 x 64 bits dual bank mode
- 数据Cache （D-Cache）
  - 8 rows of 4*64 bits in dual bank mode
  - 8 rows of 2*128 bits in single bank mode

➢ 使能开关：Flash ACR寄存器的ICEN位/ DCEN位

➢ 上电默认Cache使能状态

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBG_SWEN	Res.	Res.
													rw		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SLEEP_PD	RUN_PD	DCRST	ICRST	DCEN	ICEN	PRFTEN	Res.	Res.	Res.	Res.	LATENCY[3:0]			
	rw	rw	rw	rw	rw	rw	rw					rw	rw	rw	rw

➢ 程序实现（任选一种）：

1，使用库函数：

```
__HAL_FLASH_INSTRUCTION_CACHE_ENABLE();
__HAL_FLASH_DATA_CACHE_ENABLE();
```

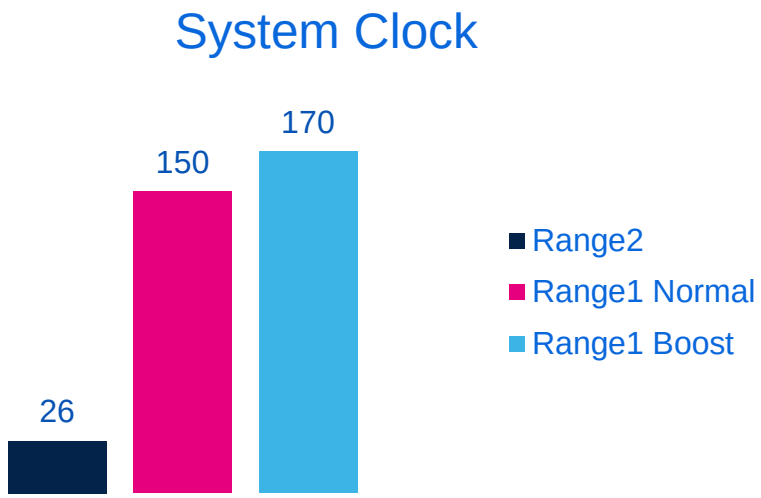
2，寄存器操作：

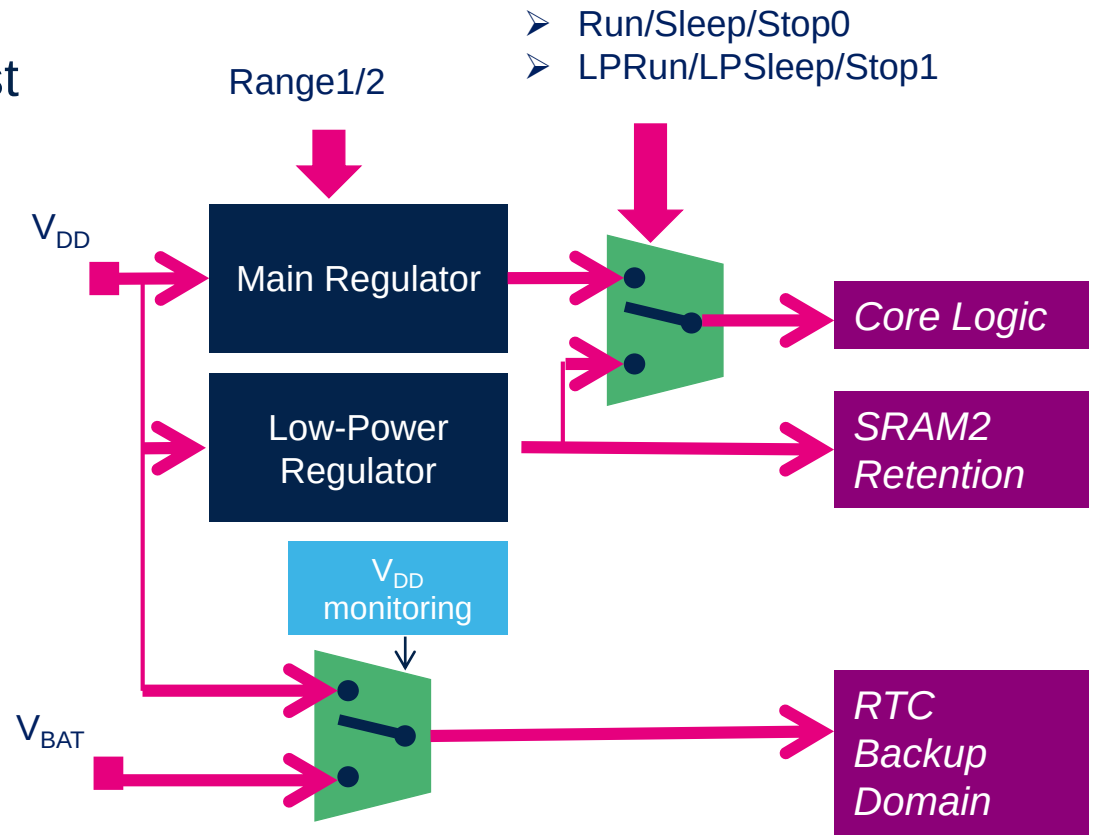
```
FLASH->ACR |= (1<<9);
FLASH->ACR |= (1<<10);
```

life.augmented

电源控制与频率

- STM32G4拥有出色的电源管理机制
- 除了具备高性能，低功耗性能同样出色
- 如果系统时钟到170MHz，需要设定为Range1 Boost

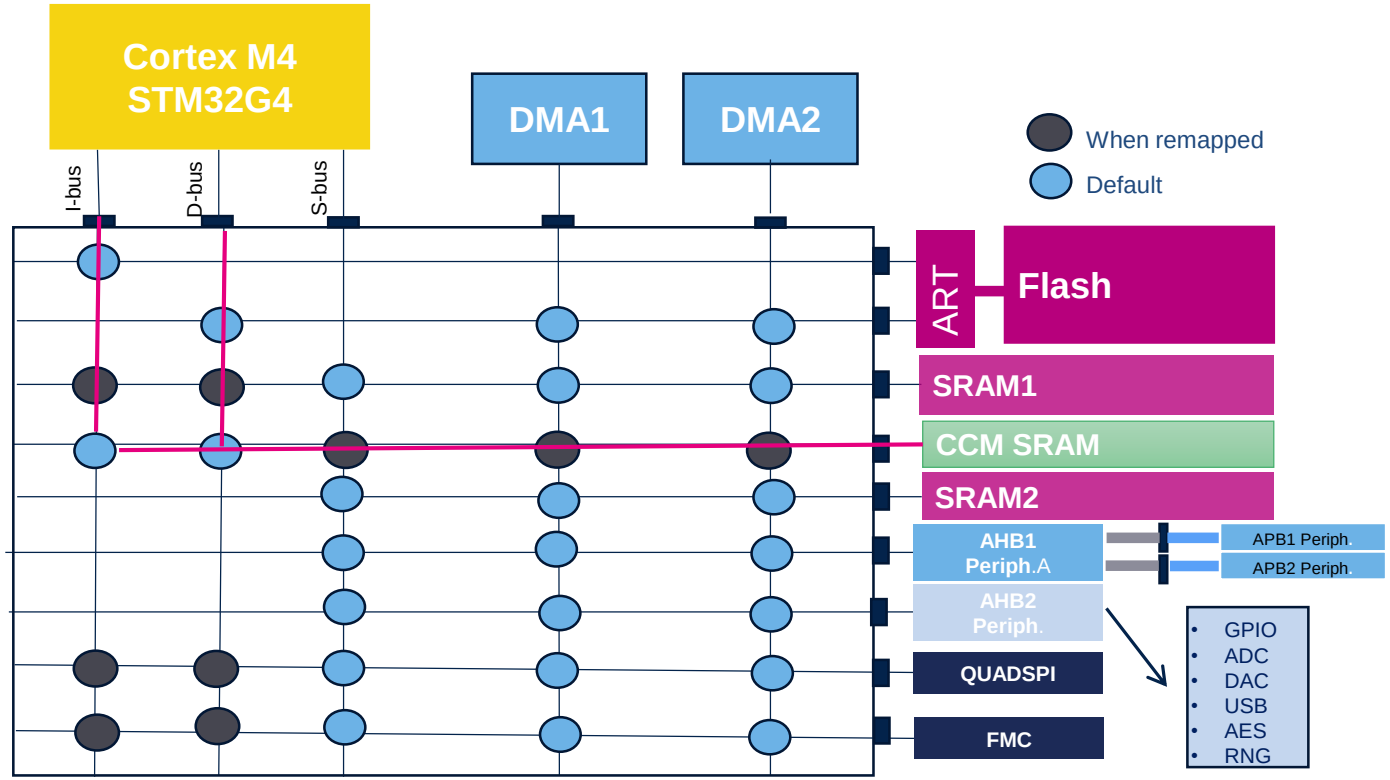




CCM SRAM加速

CCM SRAM

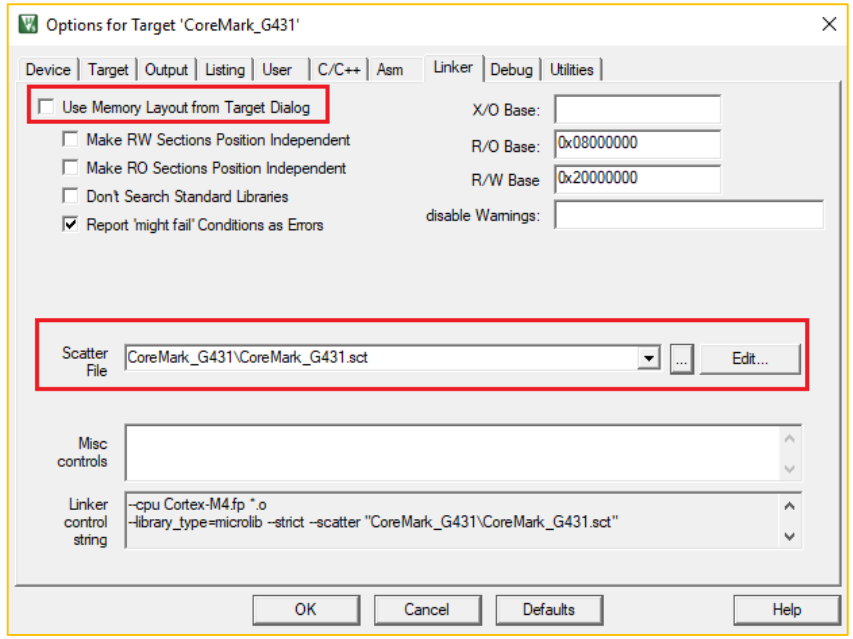
- CCM SRAM直接连接I-Bus，D-Bus
- 执行速度最快，完全发挥出170MHz的STM32G4性能
- 关键代码可放在这个区域（比如电机电流环路代码）



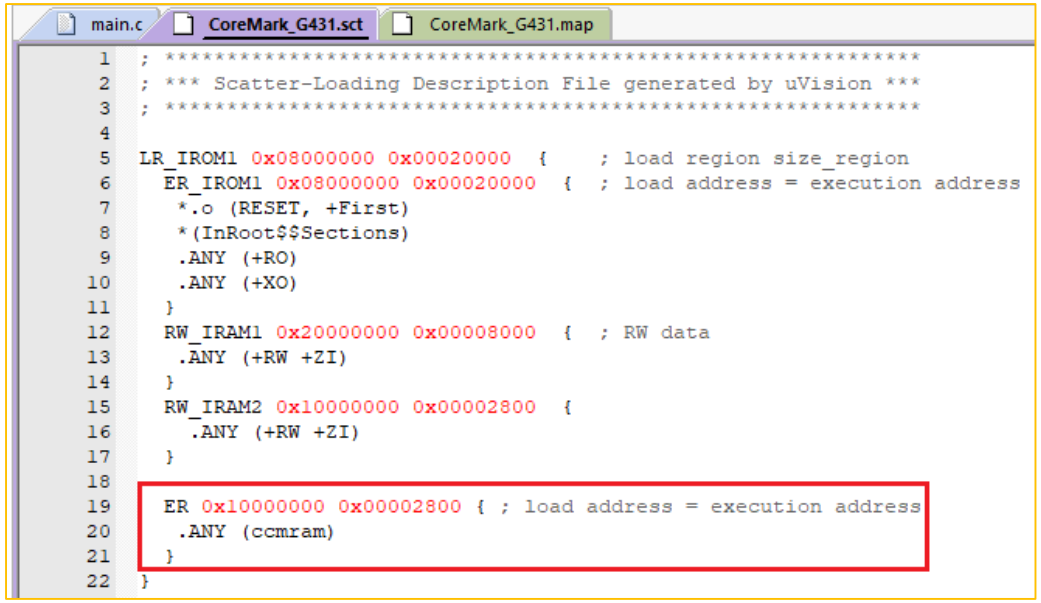
life.augmented

CCM SRAM配置举例

- Keil MDK-ARM将函数放在CCM SRAM中

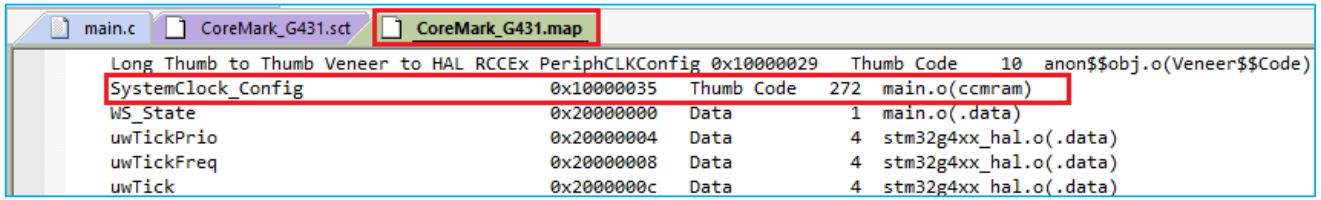


修改sct文件



函数放在CCM SRAM中

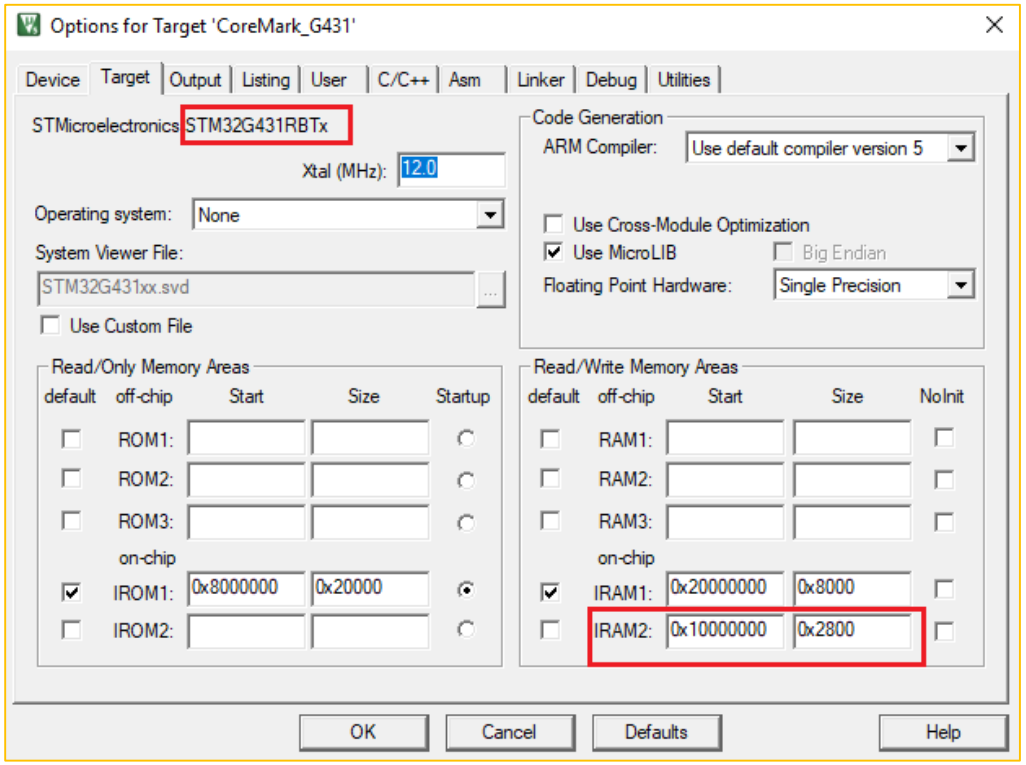
```
__attribute__((section ("ccmram")))
void SystemClock_Config(void)
{……}
```

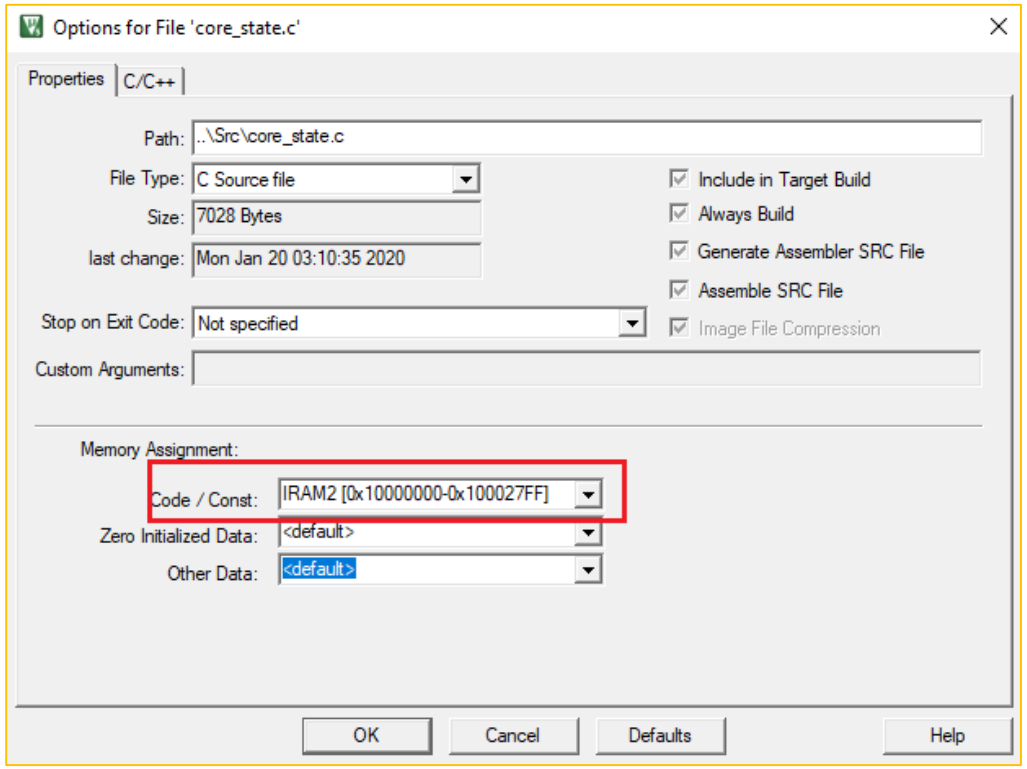


life.augmented

CCM SRAM配置举例

- Keil MDK-ARM将整个C文件放在CCM SRAM中

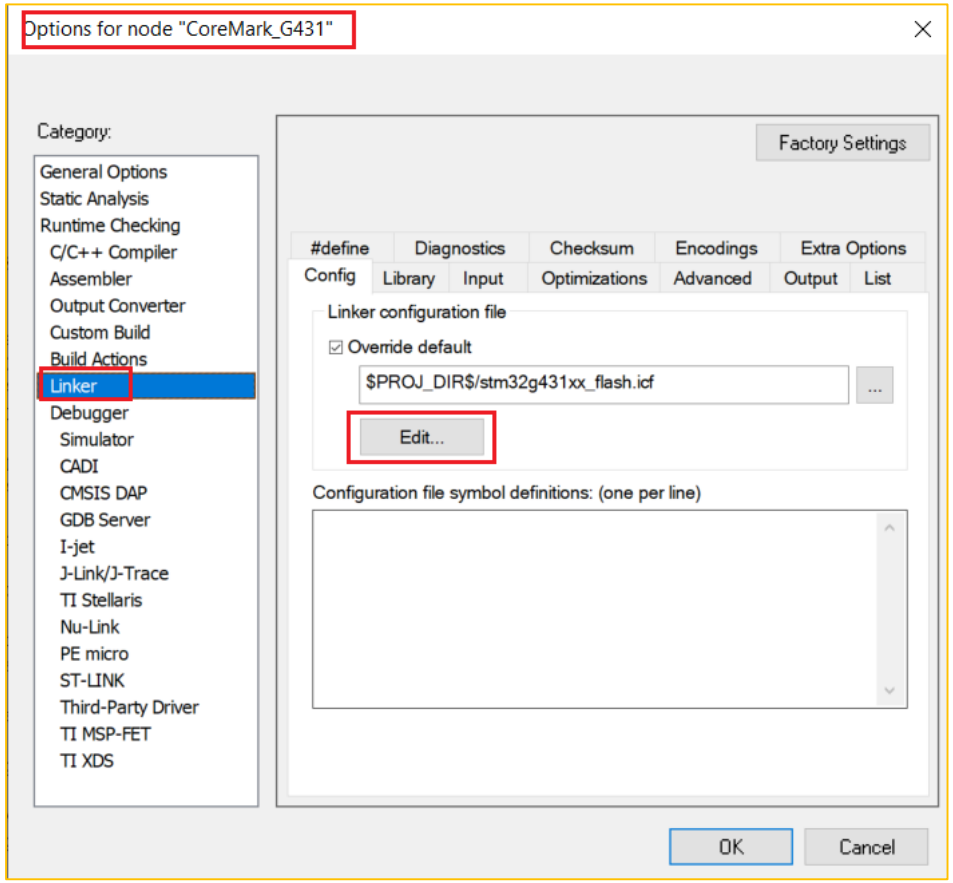




life.augmented

CCM SRAM配置举例

- IAR EWARM关于CCM SRAM修改

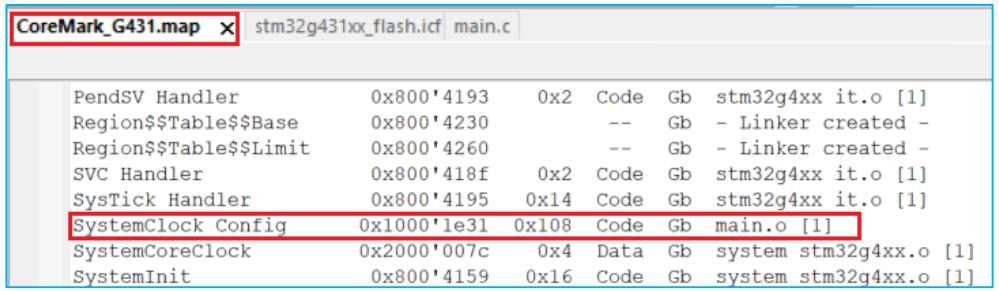


修改icf文件

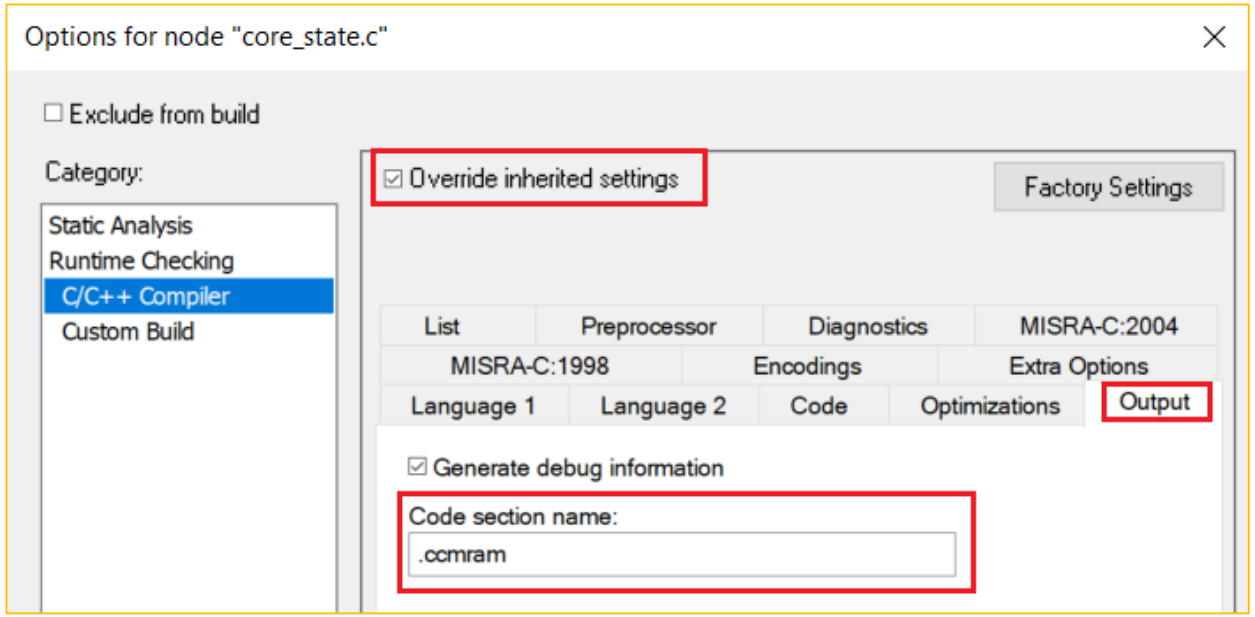
CCM SRAM配置举例

- IAR EWARM函数放在CCM SRAM中

```
#pragma location = ".ccmram"
void SystemClock_Config(void)
{……}
```



- IAR EWARM将整个C文件放在CCM SRAM中

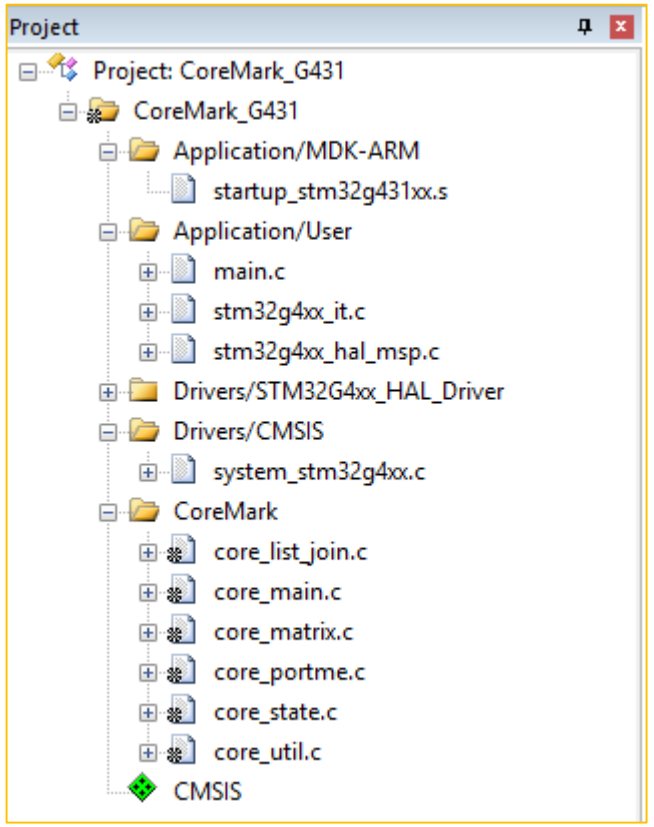


CoreMark测试 Vs 主频

CoreMark测试

- CoreMark是一项测试处理器性能的基准测试
- 代码使用C语言写成，包含：列举，数学矩阵操作和状态及CRC等运算法则
- 目前CoreMark已迅速成为测量与比较处理器性能的业界标准基准测试
- CoreMark的得分越高，意味着性能更高
- CoreMark官网的连接地址：http://www.eembc.org/coremark/index.php。

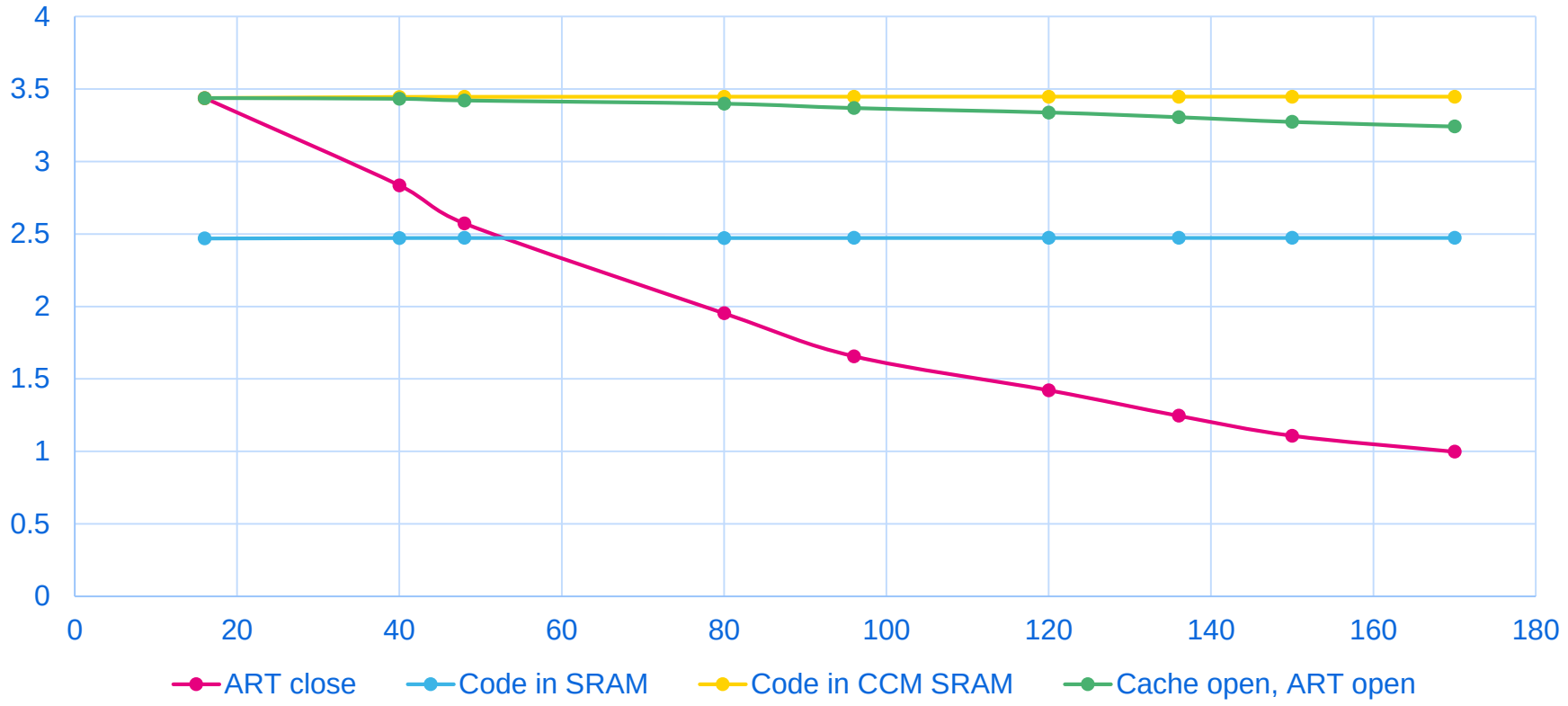
STM32G4的CoreMark测试



相同代码测试条件	主频（MHz）	CorMark分数	Cormark/MHz
程序Flash运行，ART关闭	170	169.6	0.998
程序Flash运行，ART打开	170	547.1	3.218
程序SRAM中运行	170	420.5	2.473
程序CCM SRAM运行	170	586	3.447

life.augmented

STM32G4的不同频率CoreMark测试



乘加指令 & 浮点运算

Cortex-M4 单指令周期的乘加指令

OPERATION	INSTRUCTIONS	CM3	CM4
16 x 16 = 32	SMULBB, SMULBT, SMULTB, SMULTT	n/a	1
16 x 16 + 32 = 32	SMLABB, SMLABT, SMLATB, SMLATT	n/a	1
16 x 16 + 64 = 64	SMLALBB, SMLALBT, SMLALTB, SMLALTT	n/a	1
16 x 32 = 32	SMULWB, SMULWT	n/a	1
(16 x 32) + 32 = 32	SMLAWB, SMLAWT	n/a	1
(16 x 16) ± (16 x 16) = 32	SMUAD, SMUADX, SMUSD, SMUSDX	n/a	1
(16 x 16) ± (16 x 16) + 32 = 32	SMLAD, SMLADX, SMLSD, SMLSDX	n/a	1
(16 x 16) ± (16 x 16) + 64 = 64	SMLALD, SMLALDX, SMLSLD, SMLSLDX	n/a	1
32 x 32 = 32	MUL	1	1
32 ± (32 x 32) = 32	MLA, MLS	2	1
32 x 32 = 64	SMULL, UMULL	5-7	1
(32 x 32) + 64 = 64	SMLAL, UMLAL	5-7	1
(32 x 32) + 32 + 32 = 64	UMAAL	n/a	1
32 ± (32 x 32) = 32 (upper)	SMMLA, SMMLAR, SMMLS, SMMLSR	n/a	1
(32 x 32) = 32 (upper)	SMMUL, SMMULR	n/a	1

SIMD指令调用

可以调用ARM CMSIS中的SIMD指令，实现更快运算

- Keil：cmsis_armcc.h
- IAR：cmsis_iccarm.h
- GCC: cmsis_gcc.h

cmsis_armcc.h

```
828  #define __USAD8                          __usad8
829  #define __USADA8                         __usada8
830  #define __SSAT16                         __ssat16
831  #define __USAT16                         __usat16
832  #define __UXTB16                         __uxtb16
833  #define __UXTAB16                        __uxtab16
834  #define __SXTB16                         __sxtb16
835  #define __SXTAB16                        __sxtab16
836  #define __SMUAD                          __smuad
837  #define __SMUADX                         __smuadx
838  #define __SMLAD                          __smlad
839  #define __SMLADX                         __smladx
840  #define __SMLALD                         __smlald
841  #define __SMLALDX                        __smlaldx
842  #define __SMUSD                          __smusd
843  #define __SMUSDX                         __smusdx
844  #define __SMLSD                          __smlsd
845  #define __SMLSDX                         __smlsdx
846  #define __SMLSLD                         __smlsld
847  #define __SMLSLDX                        __smlsldx
848  #define __SEL                            __sel
849  #define __QADD                           __qadd
850  #define __QSUB                           __qsub
```

C语言乘加程序

```
for(k=0; k<filtLen; k++)
{
  sum += coeffs[k] * state[stateIndex];
  stateIndex++;
}
```

使用SIMD指令

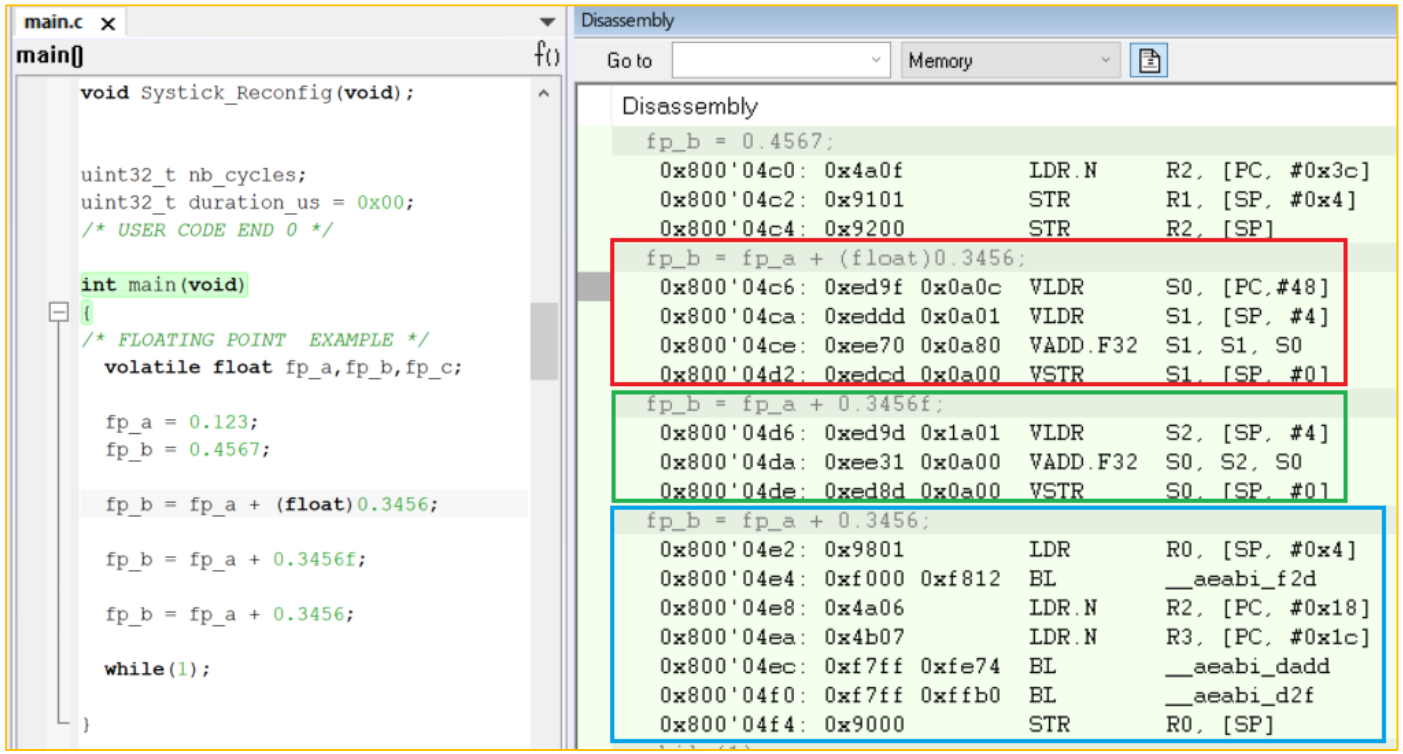
```
for(k = 0; k < filtLen; k++)
{
  c = *coeffs++;
  s = *state++;
  sum = __SMLALD(c, s, sum);
}
```

life.augmented

浮点运算指令

Operation	Description	Assembler	Cycle
绝对值数据	of float	VABS.F32	1
取反	float and multiply float	VNEG.F32 VNMUL.F32	1 1
加法	floating point	VADD.F32	1
减法	float	VSUB.F32	1
乘法	float then accumulate float then subtract float then accumulate then negate float the subtract the negate float	VMUL.F32 VMLA.F32 VMLS.F32 VNMLA.F32 VNMLS.F32	1 3 3 3 3
乘法 (带饱和限制)	then accumulate float then subtract float then accumulate then negate float then subtract then negate float	VFMA.F32 VFMS.F32 VFNMA.F32 VFNMS.F32	3 3 3 3
除法	float	VDIV.F32	14
开方	of float	VSQRT.F32	14

浮点运算示例



- STM32G4为单精度浮点
- 书写立即数建议加入float或者数据末尾加f

  fp_b = fp_a + (float)0.3456;

  fp_b = fp_a + 0.3456f;

浮点运算示例

- 浮点开根号

Keil编译器：

```
volatile float32_t Sqr_Inf1,Sqr_Inf2;
volatile float32_t Sqr_Outf[2];
Sqr_Inf1 = 1560.69f;
Sqr_Inf2 = 123.2f;
while(1)
{
    Sqr_Outf[0] = __sqrtf(Sqr_Inf1);
    Sqr_Outf[1] = __sqrtf(Sqr_Inf2);
}
```

汇编代码

```
   329:                  while(1)
   330:          {
   331:                  Sqr_Outf[0] = __sqrtf(Sqr_Inf1);
0x08000404 ED9F0A0A  VLDR          s0,[pc,#0x28]
0x08000408 ED800A01  VSTR          s0,[r0,#0x04]
0x0800040C EDD00A00  VLDR          s1,[r0,#0x00]
0x08000410 EEB10AE0  VSQRT.F32     s0,s1
0x08000414 ED810A00  VSTR          s0,[r1,#0x00]
   332:                  Sqr_Outf[1] = __sqrtf(Sqr_Inf2);
0x08000418 EDD00A01  VLDR          s1,[r0,#0x04]
0x0800041C EEB10AE0  VSQRT.F32     s0,s1
0x08000420 ED810A01  VSTR          s0,[r1,#0x04]
```

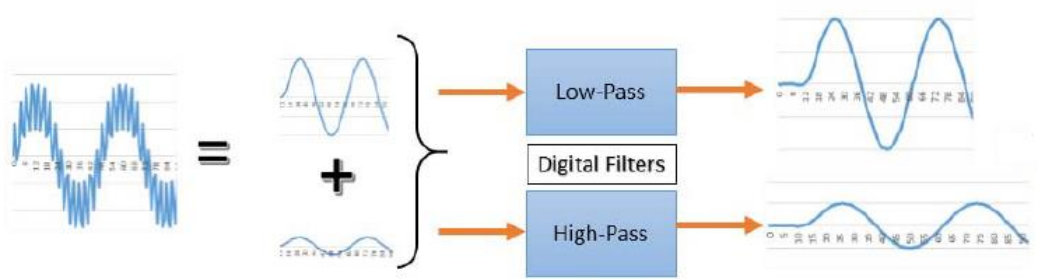
IAR编译器：

```
__ASM("VSQRT.F32 %0,%1" : "=t"(Sqr_Outf[0]) : "t"(Sqr_Inf1));
__ASM("VSQRT.F32 %0,%1" : "=t"(Sqr_Outf[1]) : "t"(Sqr_Inf2));
```

life.augmented

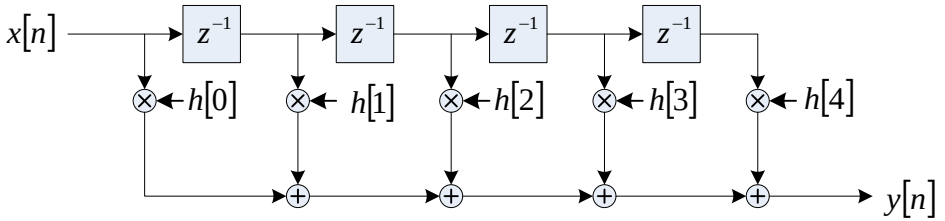
乘加指令，浮点运算举例—FIR运算

- FIR（Finite Impulse Response）滤波计算



$$y[n]=\sum_{k=0}^{N-1}h[k]x[n-k]$$

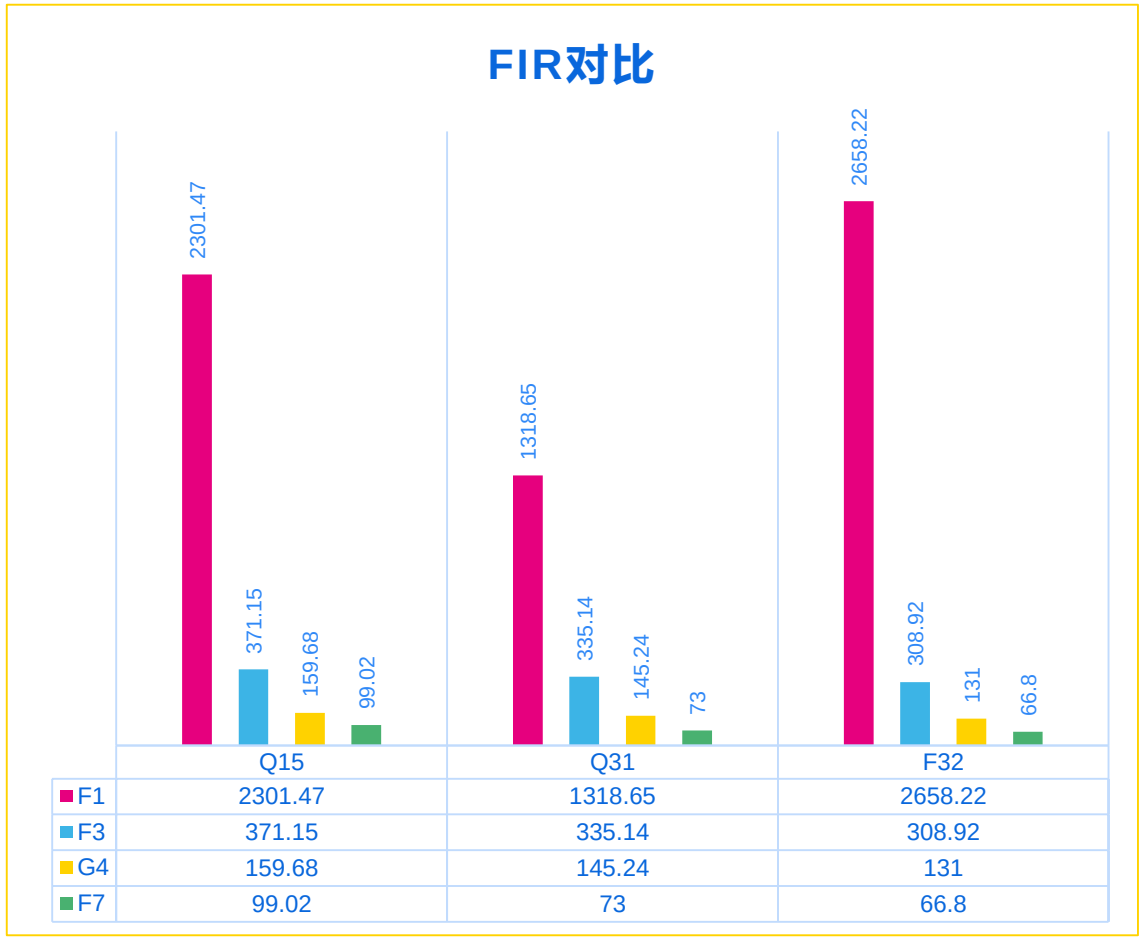
Feature / Parameter	Value
Type	Low-pass
Order	28
Sampling frequency	48 kHz
Cut-off frequency	6 kHz



life.augmented

FIR运算能力对比

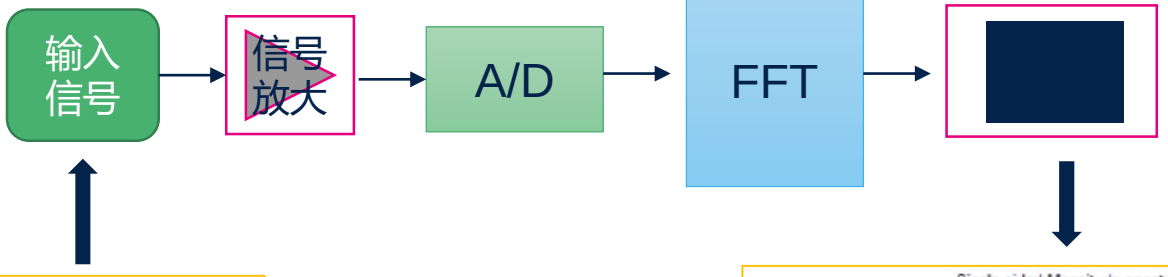
MCU	Frquency(MHz)	Cortex Core	FIR test	Cycles	Duration(us)
STM32F030	48	M0	Q15	367827	7663.06
			Q31	465986	9708.04
			F32	991849	20663.52
STM32F103	72	M3	Q15	165706	2301.47
			Q31	94943	1318.65
			F32	191392	2658.22
STM32F302	72	M4	Q15	26723	371.15
			Q31	24130	335.14
			F32	22242	308.92
STM32G431	170	M4	Q15	27146	159.68
			Q31	24691	145.24
			F32	22251	131
STM32F746	216	M7	Q15	21389	99.02
			Q31	15768	73
			F32	14429	66.8

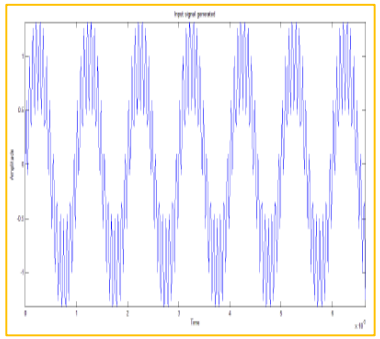


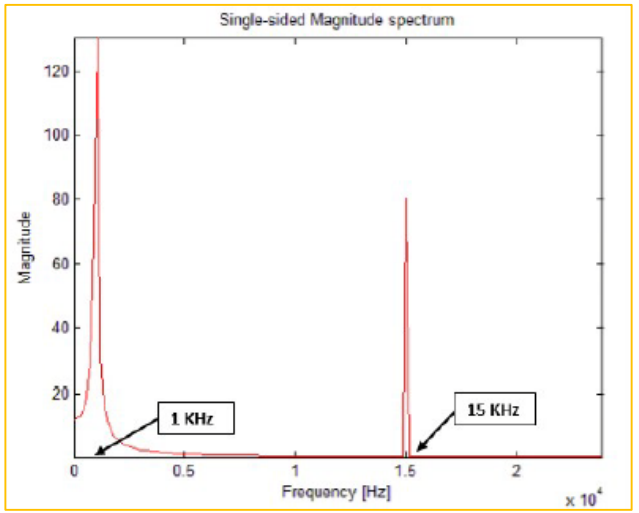
频率在内核面前将被弱化，先看内核再看频率才有意义！

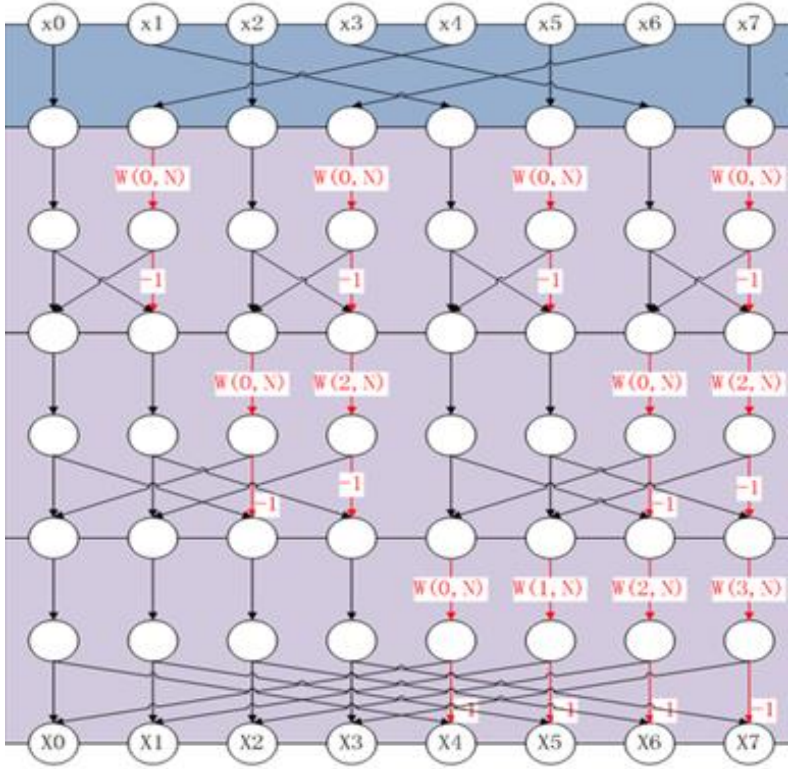
life.augmented

乘加指令，浮点运算举例— FFT运算





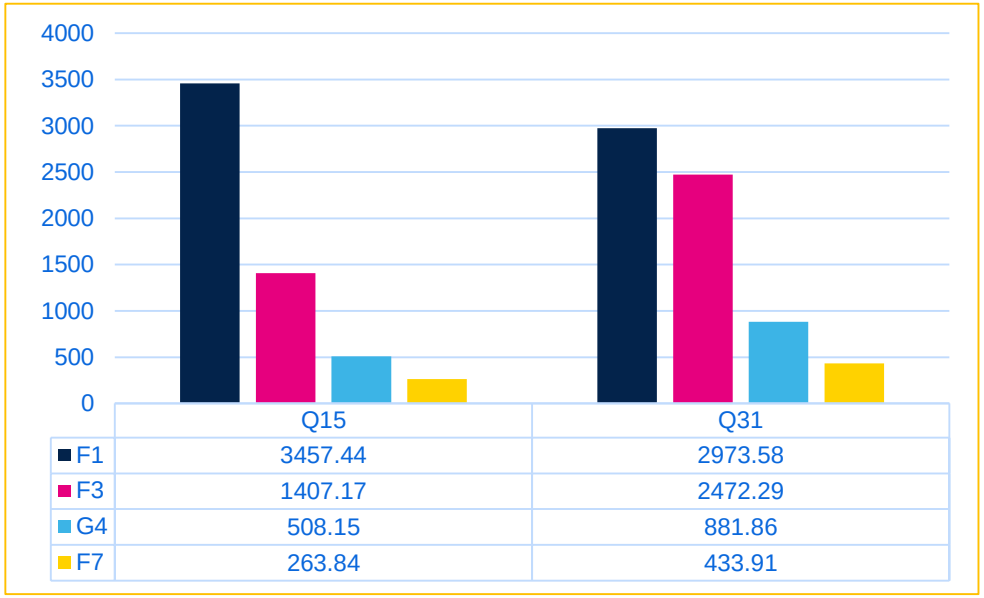




$$w_N^{nk} = e^{-j\frac{2\pi}{N}nk}$$

FFT运算能力对比

MCU	Frquency(MHz)	Cortex Core	FFT test(1024)	Cycles	Duration(us)
STM32F091	48	M0	Q15	938278	19547.46
			Q31	783106	16314.71
STM32F103	72	M3	Q15	248936	3457.44
			Q31	214098	2973.58
STM32F303	72	M4	Q15	101316	1407.17
			Q31	178005	2472.29
STM32G431	170	M4	Q15	86385	508.15
			Q31	149917	881.86
			F32	102936	606
STM32F746	216	M7	Q15	56989	263.84
			Q31	93725	433.91



life.augmented

参考文档

- **AN4841：** Digital signal processing for STM32 microcontrollers using CMSIS
- **AN4296:** CCM RAM with IAR EWARM, Keil MDK-ARM and GNU-based toolchains
- **RM0440:** STM32G4 Reference manual
- **www.stmcu.com.cn:** 如何将coremark程序移植到STM32上

Thank you

life.augmented